

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет по домашнему заданию №5

Реализация межсервисного взаимодействия посредством очередей сообщений

Выполнила:

Гайдук Алина Сергеевна

Группа K3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Нужно подключить и настроить брокер сообщений RabbitMQ и реализовать межсервисное взаимодействие через очередь сообщений в микросервисном приложении RecipeHub.

В рамках работы требуется не просто добавить контейнер RabbitMQ, а встроить асинхронный обмен событиями в бизнес-процессы: действия пользователя в engagement-service должны приводить к обновлению агрегированных счетчиков в recipe-service и blog-service без прямой синхронной записи в их базы данных.

1. подключить RabbitMQ в docker compose и передать параметры подключения сервисам;
2. создать общий контракт интеграционных событий;
3. реализовать публикацию событий при лайках и комментариях;
4. реализовать потребителей событий в сервисах рецептов и блога;
5. обеспечить надежность доставки: publisher confirms, durable queues, outbox, retry и DLQ.

Ход работы

Исходная архитектура приложения

Проект реализован как набор независимых HTTP-сервисов: gateway, identity-service, catalog-service, recipe-service, blog-service и engagement-service. Каждый сервис запускается отдельным контейнером и получает собственный DSN к базе PostgreSQL. Внешний HTTP-доступ оставлен только через gateway, а внутренние сервисы опубликованы внутри docker-сети через expose.

Для ДЗ5 ключевым стал engagement-service: он владеет действиями пользователей над контентом - лайками, сохраненными рецептами и комментариями. Recipe-service и blog-service не должны напрямую читать таблицы engagement-service для каждого списка, поэтому для счетчиков

likes_count и comments_count добавлены локальные read-model проекции, обновляемые по событиям RabbitMQ.

Компонент	Роль
RabbitMQ	Брокер сообщений; принимает integration events и маршрутизирует их по routing key.
engagement-service	Источник событий recipe.* и post.*; сохраняет бизнес-изменение и outbox-событие в одной транзакции.
recipe-service	Потребитель recipe-событий; обновляет локальные счетчики рецептов.
blog-service	Потребитель post-событий; обновляет локальные счетчики постов.
gateway	Единая внешняя точка входа для REST API; не участвует в очередях напрямую.

Контракт интеграционных событий

Контракты событий вынесены в пакет internal/domain/events. Все сообщения имеют общий Envelope: event_id, event_type, occurred_at и payload. Такой формат отделяет транспортный уровень RabbitMQ от предметной нагрузки и упрощает обработку разных типов событий одним consumer-адаптером.

Событие	Payload	Назначение
recipe.liked / recipe.unliked	target_id, user_id	Изменение количества лайков рецепта.
recipe.comment.created	target_id, comment_id, author_id	Увеличение количества комментариев рецепта.
recipe.comment.deleted	target_id, comment_id, deleted_count	Уменьшение количества комментариев рецепта с учетом удаленного поддеревя.

post.liked / post.unliked	target_id, user_id	Изменение количества лайков поста.
post.comment.created / deleted	target_id, comment_id, deleted_count	Изменение количества комментариев поста.

Публикация событий через transactional outbox

Публикация событий реализована не напрямую из HTTP-handler. Engagement-service сначала выполняет доменную операцию в PostgreSQL и в той же транзакции записывает событие в таблицу outbox_events. Например, при создании комментария сохраняется запись comments и добавляется событие recipe.comment.created или post.comment.created; при лайке сохраняется запись в таблице лайков и добавляется recipe.liked или post.liked.

Такой подход решает проблему частичной ошибки: если база записала лайк, но RabbitMQ временно недоступен, событие не теряется. Его позднее заберет outbox relay. Если транзакция откатывается, событие также не появляется, то есть внешние сервисы не получают ложное уведомление.

Outbox relay запускается внутри engagement-service. Раз в секунду он выбирает пачку до 25 непубликованных событий, помечает их claimed_at и отправляет в RabbitMQ. Выборка использует FOR UPDATE SKIP LOCKED, поэтому несколько экземпляров сервиса могут безопасно разбирать очередь outbox без двойного claim одного события.

После успешного publisher confirm событие получает published_at. При ошибке сохраняется last_error, увеличивается attempts и выставляется next_attempt_at с экспоненциальным backoff до 300 секунд. На publish также задан timeout 10 секунд, чтобы зависший confirm не блокировал relay бесконечно.

RabbitMQ client: exchange, очереди, подтверждения

RabbitMQ-клиент создает durable topic exchange `recipehub.events` и отдельный dead-letter exchange `recipehub.events.dlx`. При публикации используется JSON, DeliveryMode Persistent и `mandatory=true`. Включен confirm mode: после `PublishWithDeferredConfirmWithContext` код ожидает подтверждение брокера. Если сообщение не маршрутизируется ни в одну очередь, `NotifyReturn` превращается в ошибку публикации, и `outbox` оставляет событие для повторной попытки.

Для потребителей создаются durable очереди с `x-dead-letter-exchange`. `Recipe-service` слушает очередь `recipe-service.engagement-projections` и ключи `recipe.liked`, `recipe.unliked`, `recipe.comment.created`, `recipe.comment.deleted`. `Blog-service` аналогично слушает `blog-service.engagement-projections` и ключи `post.*`. При успешной обработке сообщение подтверждается `Ask`, при ошибке выполняется `Nack` без `requeue`, и сообщение попадает в DLQ соответствующей очереди.

Обработка событий и обновление read-model

В `recipe-service` и `blog-service` обработчики событий не выполняют сложную бизнес-логику. Они преобразуют событие в `delta` и атомарно обновляют локальную таблицу статистики: `likes_count` и `comments_count`. Для защиты от отрицательных значений используется `GREATEST(counter + delta, 0)`.

Поток для рецепта выглядит так: пользователь ставит лайк рецепту через REST API - `engagement-service` сохраняет like и `outbox` event `recipe.liked` - `>outbox` relay публикует событие в RabbitMQ - `recipe-service` получает сообщение по routing key `recipe.liked` - счетчик `likes_count` в локальной проекции рецепта увеличивается на 1. Для unlike используется `delta -1`, для удаления комментария - отрицательное значение `deleted_count`.

Для постов схема идентична, но routing keys начинаются с post. Это демонстрирует слабую связанность: engagement-service не знает деталей таблиц recipe-service и blog-service, а потребители подписываются только на интересующие их события.

Синхронные внутренние вызовы и граница применения очередей

Не все межсервисное взаимодействие переведено на очереди, и это осознанное решение. Проверки существования пользователя, рецепта или поста, а также batch-загрузка кратких карточек остаются синхронными HTTP-вызовами, потому что вызывающему use case нужен немедленный ответ. RabbitMQ используется там, где подходит eventual consistency: распространение факта изменения лайков и комментариев в read-model других сервисов.

Для снижения лишних сетевых запросов добавлены batch-вызовы: получение статистики по списку id, viewer flags по списку id и внутренний batch endpoint для кратких карточек рецептов. Это дополняет асинхронную модель: списки не делают N+1 запросы, а счетчики могут читаться из локальных проекций.

Вывод

В ходе выполнения ДЗ5 в проект был подключен RabbitMQ и реализовано асинхронное межсервисное взаимодействие на основе интеграционных событий. Engagement-service стал источником событий о лайках и комментариях, а recipe-service и blog-service - потребителями, которые обновляют собственные read-model счетчики.

Решение соответствует принципам микросервисной архитектуры: каждый сервис владеет своей базой данных, внешние запросы проходят через gateway, а распространение изменений между bounded contexts выполняется через брокер сообщений. Transactional outbox снижает риск потери событий, publisher confirms подтверждают прием сообщения брокером, а DLQ и retry/backoff делают обработку отказов управляемой.

Итоговая система поддерживает eventual consistency: пользовательская операция завершается в сервисе-владельце данных, а производные счетчики в других сервисах обновляются асинхронно. Такой подход уменьшает прямую связанность сервисов и лучше подходит для масштабирования, чем синхронное каскадное обновление нескольких баз данных.